

Informe de Arreglos de Seguridad y Calidad

Fecha: 23 de junio de 2026

Versión actual: v1.35.0

● App en producción (Dokploy / Hostinger)

Resumen ejecutivo

Durante junio realizamos una auditoría profunda y honesta de Fleet Tracker —la base ya estaba por encima del promedio: contraseñas cifradas correctamente, control de roles del lado del servidor y aislamiento entre clientes— y empezamos a tapar los agujeros que, por estar la app **ya expuesta a internet con datos reales de los pilotos**, eran explotables hoy mismo. Este informe resume, en lenguaje claro, los **catorce arreglos ya terminados**: qué riesgo concreto tenía cada uno, qué podía pasar con la versión vieja, y cómo queda protegido ahora.

Los arreglos cubren cuatro frentes. **Seguridad de acceso**: cerramos una puerta por la que un empleado de solo-lectura podía borrar empresas y equipos enteros, y cerramos la ventana de "API abierta" del primer arranque. **Integridad de los datos**: blindamos el contador de facturas para que nunca se repitan números (un problema contable serio) y frenamos manipulaciones de datos entre clientes distintos. **Estabilidad**: corregimos errores que tumbaban el Dashboard en producción y caídas intermitentes por conexiones muertas a la base. **Operaciones**: pusimos en marcha las migraciones de base de datos, los respaldos diarios, y purgamos del historial de Git un archivo con datos personales de conductores que seguía siendo recuperable.

Importante para el negocio: **todos estos arreglos se aplicaron sin sacar la app de servicio**. Los pilotos siguieron operando con normalidad. La mayoría de los cambios son invisibles para el usuario del día a día —no hay funciones nuevas que aprender—; lo que cambió es que la app es ahora más difícil de atacar, más fiable y recuperable ante un desastre.

ID	ARREGLO	SEVERIDAD	RIESGO EVITADO
SEC-1	Permisos por rol "a prueba de fallos"	CRÍTICA	Que un usuario de solo-lectura borrara empresas, equipos y cambiara integraciones.
SEC-2	Arranque inicial seguro	CRÍTICA	Que un extraño secuestrara la instancia recién desplegada antes de que tú la configures.
DATA-1	Aislamiento de escrituras entre clientes	CRÍTICA	Que un cliente marcara o modificara las alertas de otro cliente (o de todos).

ID	ARREGLO	SEVERIDAD	RIESGO EVITADO
DATA-2	Numeración de facturas a prueba de duplicados	CRÍTICA	Dos facturas con el mismo número emitidas a la vez (problema contable y legal).
OPS-1	Purga de datos de conductores del historial de Git	CRÍTICA	Que alguien recuperara el archivo con la PII de conductores del historial del repositorio.
OPS-2	Migraciones de base de datos operativas	CRÍTICA	Que un cambio de esquema tumbara silenciosamente la base en producción.
OPS-3	Respaldos automáticos de la base	CRÍTICA	Pérdida total de datos de cumplimiento (DOT/DVIR) si se cae el servidor.
DATA-6	Resúmenes y tendencias compatibles con Postgres	MEDIA	Que el Dashboard diera error 500 y no cargara en producción.
DATA-7	Conexiones a la base sin caídas por inactividad	MEDIA	Errores intermitentes (pantallazos 5xx) tras ratos de inactividad.
FE-2	App a prueba de "pantalla en blanco"	MEDIA	Que un error de pantalla dejara la app totalmente en blanco sin explicación.
SEC-3	Rate limiting + ban de IP + lockout de cuenta	ALTA	Adivinación de contraseñas por fuerza bruta y tumbar la app / disparar costos con miles de peticiones.
DATA-3	Aislamiento de usuarios entre organizaciones	ALTA	Que un admin de una empresa viera y editara los usuarios de OTRA empresa (IDOR).
DATA-4	Integridad de inventario y órdenes	ALTA	Partes duplicadas (stock partido en dos) y números de orden hija (#4.1) colisionando.
SEC-4	Sesión en cookie segura + revocación	ALTA	Robo de la sesión por XSS (token 30 días en el navegador) sin posibilidad de revocarla.

Severidad según el impacto real con la app **expuesta a internet con datos reales**: **CRÍTICA** explotable hoy / **ALTA** riesgo serio / **MEDIA** calidad y robustez.

Detalle de cada arreglo

Permisos por rol "a prueba de fallos" (RBAC fail-closed)

CRÍTICA

SEC-1

v1.30.0

QUÉ ES

El "portero" que decide qué puede hacer cada rol (administrador, mecánico, solo-lectura...). Antes, si una acción no estaba en una lista explícita, el portero la dejaba pasar por defecto.

QUÉ CAMBIÓ

El middleware de autorización (`backend/app/main.py`) pasó de *default-allow* a *fail-closed*: toda operación de escritura no contemplada queda ahora restringida a administradores. Se añadieron permisos explícitos a empresas, equipos, integraciones, POIs, reportes y DVIR.

CÓMO AFECTA LA APP ACTUAL

El administrador actual **no nota ningún cambio**: sigue pudiendo hacer todo lo que hacía. Lo que cambia es invisible para el uso normal: las cuentas con rol limitado ya no pueden hacer acciones que nunca debieron poder hacer.

✗ QUÉ PODÍA PASAR ANTES

Le dabas a un empleado una cuenta de **solo-lectura** para que viera reportes. Esa persona — o cualquiera que le robara la sesión — podía, con un poco de maña, **crear y borrar empresas y equipos enteros**, disparar importaciones del ELD y hasta cambiar el proveedor de GPS activo. Tu "observador" tenía, en la práctica, manos de administrador en media app.

✓ QUÉ PASA AHORA

Cualquier acción de escritura sin permiso explícito se **rechaza con un error de "no autorizado"**. Solo-lectura es de verdad solo-lectura. Las acciones sensibles quedaron limitadas a administradores salvo que se les conceda el permiso concreto.

Arranque inicial seguro (bootstrap)

CRÍTICA

SEC-2

v1.30.0

QUÉ ES

El estado de la app justo después de instalarse, cuando todavía no hay ningún usuario creado y estás a punto de configurar tu cuenta de administrador.

QUÉ CAMBIÓ

Mientras la tabla de usuarios esté vacía, la API ya no queda abierta: sin sesión válida responde "no autenticado" (401), salvo una lista corta de rutas necesarias (estado, configuración inicial, login, salud, marca). El asistente de configuración sigue funcionando (`backend/app/main.py`).

CÓMO AFECTA LA APP ACTUAL

Ninguno visible. La app ya está configurada y en producción; este blindaje protege el momento del primer arranque y de futuros redespiegues desde cero.

✗ QUÉ PODÍA PASAR ANTES

En el instante en que el servidor levantaba la app por primera vez y **aún no existía ningún usuario**, la API quedaba **completamente abierta**. Entre que el contenedor arrancaba y tú completabas el asistente de registro, un extraño que estuviera observando el servidor podía **adelantarse y crearse a sí mismo como el administrador** de tu Fleet Tracker. Te quedabas fuera de tu propia app.

✓ QUÉ PASA AHORA

Sin usuarios, la API solo deja pasar lo imprescindible para que **tú** completes el registro inicial; todo lo demás está cerrado. La ventana de secuestro del primer arranque queda eliminada.

Aislamiento de escrituras entre clientes (IDOR de escritura)

CRÍTICA

DATA-1

v1.30.0

QUÉ ES

La barrera que evita que los datos de un cliente se mezclen con los de otro. La barrera protegía bien las *lecturas*, pero una operación de *escritura* sobre las alertas se le escapaba.

QUÉ CAMBIÓ

La función que marca alertas como atendidas (`ack_events` en `backend/app/core/alerts.py`) hacía un `UPDATE` sin filtrar por organización. Ahora filtra siempre por la organización del cliente actual.

CÓMO AFECTA LA APP ACTUAL

Invisible. Cada cliente sigue viendo y gestionando solo sus alertas, pero ahora también a nivel de escritura está garantizado por código.

✗ QUÉ PODÍA PASAR ANTES

Un usuario de la empresa A, manipulando una petición, podía marcar como "atendidas" las **alertas de la empresa B**. Peor: una llamada sin especificar cuáles, marcaba como atendidas **las alertas de TODAS las empresas de la base** a la vez. Alguien podía "apagar" las alertas de cumplimiento de otro cliente sin que este se enterara.

✓ QUÉ PASA AHORA

Cada operación sobre alertas queda **amarrada a la organización del usuario** que la ejecuta. Es imposible tocar las alertas de otro cliente, y mucho menos las de todos.

Numeración de facturas a prueba de duplicados (contador atómico)

CRÍTICA

DATA-2

v1.30.1

QUÉ ES

El mecanismo que asigna el número correlativo a cada factura del taller (1, 2, 3...) para que nunca se repita.

QUÉ CAMBIÓ

El contador anterior leía el número, sumaba uno y lo guardaba sin candado, dejando una rendija para el solapamiento. Se reemplazó por `db.bump_invoice_number` con bloqueo a nivel de base (`SELECT ... FOR UPDATE`), atómico en Postgres. Sin cambios de esquema.

CÓMO AFECTA LA APP ACTUAL

Invisible y sin cambios de comportamiento: las facturas se siguen numerando igual, pero ahora es matemáticamente imposible repetir un número aunque se facture en paralelo.

✗ QUÉ PODÍA PASAR ANTES

Si dos personas facturaban dos órdenes de trabajo **en el mismo instante** —cosa normal en un taller con varios mostradores—, ambas podían tomar el **mismo número de factura**. Resultado: dos facturas distintas con el número 1043. Eso es un **problema contable y legal** que ensucia tu contabilidad y complica auditorías o reclamos.

✓ QUÉ PASA AHORA

La base de datos entrega el número con un candado: la segunda factura **espera su turno** y recibe el siguiente número. Nunca más dos facturas con la misma numeración.

Respaldos automáticos de la base de datos

CRÍTICA

OPS-3

v1.30.1

QUÉ ES

Copias de seguridad periódicas de toda la información de la app (órdenes, facturas, inventario, mantenimiento, DVIR) para poder recuperarla si algo sale mal.

QUÉ CAMBIÓ

Se añadió `ops/backup.sh` (vuelca la base con `pg_dump`, comprimido, con retención de 14 días, pensado para correr automáticamente en el servidor) más un manual de restauración (`ops/db-restore.md`) y la sección 4.1 en `DEPLOY.md`.

CÓMO AFECTA LA APP ACTUAL

Invisible para el usuario. Aporta una red de seguridad: ante un fallo, los datos se pueden restaurar. *Pendiente:* dejar el respaldo programado en el servidor por SSH y copiar las copias fuera del propio servidor.

✗ QUÉ PODÍA PASAR ANTES

No existía **ningún respaldo**. La única "protección" era no borrar el disco. Si el VPS de Hostinger sufría una falla de hardware, un borrado accidental o un ataque, se perdían **de golpe y para siempre** todas las órdenes de trabajo, facturas, inventario y —lo más grave— los registros de cumplimiento DOT/DVIR que la ley obliga a conservar.

✓ QUÉ PASA AHORA

Existe un script de respaldo comprimido con 14 días de retención y un manual de restauración paso a paso. Ante un desastre, los datos se recuperan en lugar de perderse.

Migraciones de base de datos operativas (Alembic)

CRÍTICA

OPS-2

v1.31.0 / v1.31.1

QUÉ ES

El sistema que aplica de forma ordenada y segura los cambios en la *estructura* de la base de datos (añadir una columna, una tabla nueva...) cada vez que sale una versión nueva.

QUÉ CAMBIÓ

El contenedor ya no depende de `create_all` para el esquema. Un nuevo `backend/scripts/db_migrate.py` corre al arrancar: en base existente sin historial la **adopta** sin tocar datos y aplica los cambios pendientes; en base nueva la crea desde cero. Se regeneró el esquema base completo (19 tablas, antes faltaban 3). En la v1.31.1 se corrigió un fallo donde Alembic **crasheaba en producción** por el carácter `%` de la contraseña de Postgres (entraba en bucle de reinicio); ahora la URL se pasa directa al motor.

CÓMO AFECTA LA APP ACTUAL

Invisible en uso normal, pero clave para el futuro: **desbloquea** poder publicar mejoras que cambian la estructura de datos sin riesgo de romper la base en silencio.

✗ QUÉ PODÍA PASAR ANTES

Como cada cambio se desplegaba automáticamente en cuanto subías código, y el sistema viejo **no sabía modificar tablas que ya existían**, bastaba una mejora que añadiera una columna para que la app **se rompiera en producción de forma silenciosa**: error 500 al usar la pantalla afectada, sin aviso claro, sobre la base real de los pilotos. Tú te enterabas por la queja del usuario, no por una alerta.

✓ QUÉ PASA AHORA

Al arrancar, la app aplica los cambios de estructura de forma **ordenada y verificada** (probado en los tres escenarios: base nueva, base existente adoptada y base ya migrada). Los cambios de esquema futuros dejan de ser una ruleta rusa.

Purga de datos de conductores del historial de Git

CRÍTICA

OPS-1

limpieza de repositorio

QUÉ ES

El historial de Git guarda **todas** las versiones pasadas de cada archivo. Un archivo con datos personales de conductores (`roster.csv`) se había borrado del proyecto, pero seguía vivo y recuperable en el historial.

QUÉ CAMBIÓ

Se reescribió el historial completo con `git filter-branch` para eliminar `roster.csv` de **todos** los commits, y se hizo `force-push` de la rama `main` y de las 67 etiquetas (tags). El archivo deja de existir en cualquier punto del historial.

CÓMO AFECTA LA APP ACTUAL

Ninguno funcional. Es higiene de privacidad y cumplimiento: la PII de conductores ya no es extraíble del repositorio.

✗ QUÉ PODÍA PASAR ANTES

Aunque el archivo "ya no estaba" en el proyecto, cualquiera con acceso al repositorio podía **recuperar el archivo de datos de conductores** con un solo comando, viajando al commit antiguo. Quedaban expuestos nombres y datos personales de conductores de un empleador anterior: un incidente de privacidad y un posible problema legal latente.

✓ QUÉ PASA AHORA

El archivo fue **borrado de raíz de todo el historial** y el cambio se forzó en el repositorio remoto (rama y 67 etiquetas). Ya no hay forma de recuperar esa información desde Git.

Resúmenes y tendencias compatibles con Postgres

MEDIA

DATA-6

v1.30.0

QUÉ ES

Los cálculos que arman el resumen del mes, las tendencias y el listado de conductores faltantes —lo que alimenta el Dashboard.

QUÉ CAMBIÓ

Esos cálculos usaban una función de fecha (`strftime`) que solo existe en SQLite, no en la base de producción (Postgres). Se reemplazó por un rango de fechas portable (`_month_bounds` en `backend/app/db.py`) que funciona en ambas.

CÓMO AFECTA LA APP ACTUAL

El usuario nota que **el Dashboard y los resúmenes vuelven a cargar** correctamente en producción, sin errores.

✗ QUÉ PODÍA PASAR ANTES

En desarrollo todo funcionaba (porque allí se usa otra base de datos), pero en producción el **panel de resumen del Dashboard daba error 500 y no cargaba**. El usuario abría su pantalla principal y se encontraba con un fallo en vez de sus números de la flota.

✓ QUÉ PASA AHORA

Los cálculos de resumen, tendencias y faltantes usan fechas compatibles con Postgres y **cargan sin error** tanto en desarrollo como en producción.

Conexiones a la base sin caídas por inactividad

MEDIA

DATA-7

v1.30.0

QUÉ ES

La forma en que la app mantiene abiertas las conexiones a la base de datos para responder rápido.

QUÉ CAMBIÓ

Se configuró el motor con `pool_pre_ping=True` (verifica que la conexión siga viva antes de usarla) y `pool_recycle=1800` (renueva conexiones cada 30 minutos), evitando los errores de `SSL connection closed`.

CÓMO AFECTA LA APP ACTUAL

El usuario nota **menos errores intermitentes**, sobre todo al volver a la app tras un rato sin usarla.

✗ QUÉ PODÍA PASAR ANTES

Tras un rato de inactividad, la base cerraba la conexión por su cuenta pero la app no se daba cuenta. La siguiente acción del usuario —abrir una pantalla, guardar algo— se topaba con una **conexión muerta y devolvía un error 5xx**. El usuario tenía que recargar y volver a intentarlo, sin saber por qué había fallado.

✓ QUÉ PASA AHORA

La app **comprueba y renueva** las conexiones automáticamente. Las caídas intermitentes por conexiones envejecidas desaparecen.

App a prueba de "pantalla en blanco" (Error Boundary)

MEDIA

FE-2

v1.30.0

QUÉ ES

Una red de seguridad en la interfaz para que, si una pantalla falla al dibujarse, la app no se quede totalmente en blanco.

QUÉ CAMBIÓ

Se envolvió la app con un `ErrorBoundary` que captura los errores de render y muestra un mensaje en vez de una página vacía. Además, se corrigió un comentario de seguridad **falso** en `LoginPage.tsx` que afirmaba —erróneamente— que no había autenticación real (sí la hay).

CÓMO AFECTA LA APP ACTUAL

Si algo falla en una pantalla, el usuario ve un **mensaje claro con opción de recargar** en lugar de quedarse mirando una página en blanco sin saber qué pasó.

✗ QUÉ PODÍA PASAR ANTES

Bastaba un único error inesperado al dibujar una pantalla para que **toda la app se pusiera en blanco**, sin mensaje ni pista. El usuario asumía que "la app se rompió" o "se cayó internet", cerraba y, a veces, te llamaba alarmado. Además, un comentario en el código mentía sobre la seguridad del login y podía confundir a quien lo mantuviera.

✓ QUÉ PASA AHORA

Un fallo de pantalla se contiene y muestra "Algo salió mal — Recargar" sin tumbar la app entera. Y la documentación de seguridad del login refleja la realidad.

Rate limiting + ban de IP + lockout de cuenta (anti fuerza bruta)

ALTA

SEC-3

v1.32.0

QUÉ ES

El freno que limita cuántas veces seguidas se puede intentar iniciar sesión. Sirve para que nadie pueda "probar y probar" contraseñas a toda velocidad, ni bombardear la app con miles de peticiones hasta tumbarla.

QUÉ CAMBIÓ

Nuevo `backend/app/core/ratelimit.py` (en memoria, thread-safe; lee la IP real desde `X-Forwarded-For` tras Traefik). El login aplica, en orden: ban de IP → límite de **5 intentos/min por IP** → bloqueo de cuenta → credenciales. Tras 15 fallos en 10 min la IP queda **baneada 15 min**; tras 8 fallos la cuenta se **bloquea** 15 min. Responde `429 + Retry-After`. Configurable por env `SEC3_*` (valores por defecto sanos). Incluye el primer test del repo (`test_ratelimit.py`).

CÓMO AFECTA LA APP ACTUAL

Para un usuario legítimo es **invisible**: nadie teclea su contraseña cinco veces por minuto. Solo lo nota quien intente abusar del login, que recibe un "demasiados intentos".

✗ QUÉ PODÍA PASAR ANTES

El login **no tenía ningún freno**. Alguien que conociera la URL podía **probar miles de contraseñas** (fuerza bruta) hasta dar con una, o **tumbar la app y disparar los costos** lanzándole mil peticiones por segundo. Nada lo detenía ni lo ralentizaba.

✓ QUÉ PASA AHORA

Máximo 5 intentos por minuto y por IP; tras 15 fallos la IP queda baneada 15 minutos y tras 8 fallos la cuenta se bloquea. La app responde **429 "demasiados intentos"** y corta el ataque, incluso si viene repartido entre varias IPs.

Aislamiento de usuarios entre organizaciones (IDOR)

ALTA

DATA-3

v1.33.0

QUÉ ES

La barrera que evita que un administrador de una empresa pueda meterse con las cuentas de usuario de OTRA empresa. La lista y edición de usuarios se le escapaba a esa barrera.

QUÉ CAMBIÓ

Las funciones `list_users` y `update_user` ahora filtran y validan por la organización del request (el modelo `User` no es `OrgScoped`, así que sus consultas no se filtraban solas).

Diferido: username único **por organización**, que requiere rediseñar el login para desambiguar.

CÓMO AFECTA LA APP ACTUAL

Invisible. Cada administrador sigue gestionando solo los usuarios de su empresa, pero ahora está garantizado por código que no puede tocar los de otra.

✗ QUÉ PODÍA PASAR ANTES

Un administrador de una empresa podía **ver y editar los usuarios de OTRA empresa** manipulando una petición (un fallo de tipo IDOR). Además, el nombre de usuario era **único a nivel global**: si una empresa registraba "admin", la siguiente ya no podía usar ese nombre — rompiendo la idea de tener varios clientes independientes en la misma app.

✓ QUÉ PASA AHORA

La gestión de usuarios queda **amarrada a la organización** de quien la ejecuta. Un administrador solo ve y edita los usuarios de su propia empresa; los de los demás clientes le son invisibles e intocables.

Integridad de inventario y órdenes (reglas de unicidad)

ALTA

DATA-4

v1.33.0

QUÉ ES

Las reglas que impiden que se cree dos veces la misma parte del catálogo o que dos órdenes de trabajo enlazadas reciban el mismo número de "hija" (#4.1).

QUÉ CAMBIÓ

Se añadieron restricciones de unicidad **a nivel base de datos**: `UniqueConstraint(org_id, part_number)` en las partes y `(parent_id, child_seq)` en las órdenes. `create_part` y `create_wo` capturan el error de integridad y fallan con un mensaje claro en vez de duplicar en silencio. Es la **primera migración Alembic versionada** aplicada en producción (migración `1048cd72c14a`, encadenada al baseline).

CÓMO AFECTA LA APP ACTUAL

Invisible en uso normal. Aporta una garantía de fondo: ni el catálogo de partes ni la numeración de órdenes se pueden ensuciar, ni siquiera con clics simultáneos.

× QUÉ PODÍA PASAR ANTES

Dos acciones **en el mismo instante** podían **duplicar una parte** del catálogo —su stock quedaba partido en dos filas distintas, descuadrando el inventario— o **colisionar el número de una orden hija (#4.1)**, dejando dos órdenes con la misma numeración. La app comprobaba "¿ya existe?" justo antes de crear, pero esa rendija dejaba colarse los duplicados.

✓ QUÉ PASA AHORA

La propia base de datos rechaza el duplicado y la app responde con un **error claro** en lugar de crear una segunda copia. El inventario y la numeración de órdenes quedan blindados.

Sesión en cookie segura + revocación

ALTA

SEC-4

v1.34.0 / v1.35.0

QUÉ ES

El "pase" que demuestra que ya iniciaste sesión y que la app revisa en cada acción. El problema era dónde se guardaba ese pase y que no había forma de anularlo.

QUÉ CAMBIÓ

El login ahora deja el token en una **cookie** `HttpOnly` + **Secure** + **SameSite=Strict** que el JavaScript no puede leer; el middleware la lee (con respaldo a `Bearer`). El token incluye un `token_version` (columna nueva, migración `df63703c9a5b`): cambiar la contraseña lo incrementa e **invalida los tokens viejos**. Hay un endpoint real `/auth/logout`. En la v1.35.0 el frontend dejó de guardar el token en `localStorage` y se apoya solo en la cookie + `/auth/status`.

CÓMO AFECTA LA APP ACTUAL

Casi invisible: al desplegar, las sesiones viejas dejan de valer y hay que **volver a iniciar sesión una vez**. A partir de ahí, cerrar sesión y cambiar la contraseña funcionan de verdad.

✗ QUÉ PODÍA PASAR ANTES

El pase de sesión vivía **30 días guardado en el navegador (localStorage), legible por cualquier script**. Un ataque de tipo XSS podía **robar la sesión** y suplantar al usuario durante un mes. Y no había forma de revocarla: **cambiar la contraseña no cerraba las sesiones viejas**, así que un pase robado seguía valiendo.

✓ QUÉ PASA AHORA

La sesión viaja en una **cookie que el JavaScript no puede leer** (a salvo de robo por XSS), cambiar la contraseña **invalida las sesiones viejas** y existe un logout real que cierra la sesión de verdad.

Qué queda por hacer

Estos catorce arreglos cubren lo crítico y lo de riesgo alto explotable de inmediato —incluida ahora toda la capa de autenticación (login con freno, sesión en cookie segura) y el aislamiento entre clientes. El plan de blindaje (`ROADMAP-SEGURIDAD.md`) continúa por etapas hacia el estándar OWASP ASVS Nivel 2, propio de apps con datos personales y financieros. Lo principal que sigue en cola:

- **SEC-5 / SEC-6 — Endurecimiento adicional:** cabeceras de seguridad (CSP/HSTS) y rotación del secreto que firma las sesiones.
- **DATA-5 / N+1 — Rendimiento:** optimizar las consultas de reportes y listados pesados para que escalen sin cargar de más en memoria.
- **ARCH-1 — Multi-réplica:** mover los trabajos en cola y el loop de alertas fuera de la memoria (a Postgres/redis) para poder correr varias instancias de la app.

- **OPS-4 / OPS-5 — Operaciones:** que el contenedor no corra como administrador (no-root) y una integración continua (CI) que frene despliegues con errores.
- **Más adelante:** segundo factor (MFA) para administradores, y el panel de administración / CRM de suscripciones.

Nota operativa: todos los arreglos descritos en este informe se aplicaron con la app **operativa en producción**. Los pilotos siguieron usando Fleet Tracker con normalidad durante todo el proceso; los despliegues fueron incrementales y, salvo el candado HTTPS aún pendiente, no hubo interrupción del servicio.